

Etude CPU complète: Aspirateur / Integrate

Analyse focalisée sur la réduction du CPU dans le pipeline de collecte et d'intégration PmaControl, avec gains mesurés sur la première vague d'optimisation transparente.

Repo: /srv/www/pmacontrol

Branche analysée: cpu-wave-1-aspirateur-collect

Base: commercial

Date: 2026-04-15

RUNS BENCH

1000

micro-bench ciblé sur les optimisations déjà appliquées

GAIN CACHE SHOW DATABASES

x6.95

sur la même connexion

RÉDUCTION APPELS SHOW DATABASES

99.9%

1000 appels → 1 appel

SUPPRESSION PARSE_INI MORT

x23538

gain pur CPU sur 1000 runs

Conclusion exécutive

Le CPU du pipeline n'est pas consommé par une seule fonction isolée. Il part surtout dans les multiplicateurs: trop de collectes larges, trop de transformations répétées, trop de sérialisations JSON, trop de normalisations de tableaux, et trop de petits payloads qui repassent dans `Integrate::evaluate()`.

Le levier principal n'est donc pas le micro-tuning ligne par ligne. Le vrai levier est de faire moins de travail redondant, à sortie identique. Cette étude sépare trois vagues de travail pour garder un risque maîtrisé.

Vagues proposées

- **Wave 1:** optimisations CPU transparentes dans `Aspirateur`, sans changer fréquence ni données collectées.
- **Wave 2:** réduction du CPU pur PHP dans `Integrate` sur les parcours, typages, échappements et caches metadata.
- **Wave 3:** optimisation du pipeline complet via batching et réduction de la fragmentation des payloads.

Périmètre et contrainte

La demande explicite a été respectée pour la branche courante:

- **Conservé** même contenu collecté
- **Conservé** même fréquence de collecte
- **Conservé** même format fonctionnel des exports
- **Ciblé** uniquement le CPU perdu dans du travail redondant ou mort

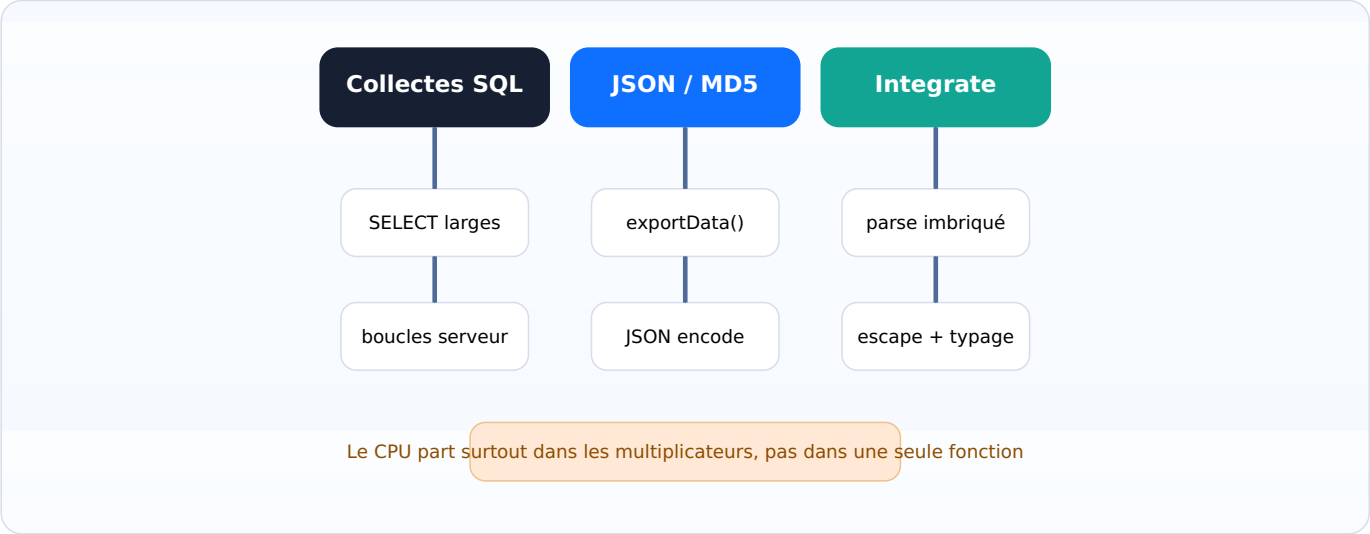


Objectif du chantier CPU: réduire le travail redondant sans changer le comportement métier

Hotspots CPU identifiés

ZONE	FICHER	CAUSE CPU	TYPE	PRIORITÉ
Collectes larges metadata	Aspirateur.php	`information_schema.tables`, `schemata`, `INNODB_METRICS`, `processlist`, `events_statements_summary_by_digest`	collecte	haute
Export morcelé	Aspirateur.php	multiples `exportData()` par run, avec JSON + MD5 + pivot file	pipeline	haute
Normalisation profonde	Integrate.php	`array_change_key_case()`, parcours imbriqués, typage et échappement à chaud	parse PHP	haute
Concaténation SQL	Integrate.php	construction de gros `INSERT ... VALUES (...)` en PHP	CPU PHP	moyenne
Metadata rechargées	Integrate.php	`ts_file`, `ts_variable`, checks de types rejoués inutilement	cache	moyenne

Schéma logique des pertes CPU



Benchmark Wave 1

MESURE	AVANT	APRÈS	GAIN
Cache `SHOW DATABASES`	5.423 ms	0.780 ms	x6.95
Appels `SHOW DATABASES`	1000	1	-99.9%
Bloc mort `parse_ini_file`	362.592 ms	0.015 ms	x23538

Lecture du résultat

La première vague ne change ni la fréquence ni les données collectées. Elle retire uniquement du travail CPU inutile ou rejoué à l'identique.

Visualisation relative



Découpage recommandé par branches

BRANCHE	OBJET	GAINS ATTENDUS	RISQUE
cpu-wave-1-aspirateur-collect	retirer le CPU redondant ou mort dans `Aspirateur`, sans changer la collecte	faible à modéré mais sûr	bas
cpu-wave-2-integrate-core	réduire le coût de `Integrate::evaluate()` et des transformations profondes	élevé	moyen
cpu-wave-3-pipeline-batching	réduire le nombre de payloads, `exportData()` et transformations répétées	très élevé	élevé

Plan de réduction CPU

Wave 1 - transparent

- supprimer le travail mort
- cacher les metadata relues dans le même cycle
- éviter les rescans inutiles

Wave 2 - Integrate

- éviter les `array\_change\_key\_case()` multiples
- cacher `ts\_file` et `ts\_variable`
- réduire `getTypedData()` / `isJson()` sur les chemins chauds
- réduire les concaténations et échappements répétés

Wave 3 - pipeline

- réduire le nombre de `exportData()` par run serveur
- réduire la fragmentation des payloads
- envoyer des blocs plus cohérents à `Integrate`

Règles à respecter

- ne pas changer la donnée métier sans décision explicite
- ne pas changer la fréquence de collecte sur les branches transparentes
- mesurer avant / après avec micro-bench et test ciblé
- séparer les gains sûrs des gains structurels plus risqués

Principe directeur

Wave 1: même comportement, moins de CPU

Wave 2: même sortie, parse plus efficace
Wave 3: moins de transformations au global

Fichiers principaux du chantier

FICHIER	RÔLE	OBSERVATION
App/Controller/Aspirateur.php	collecte et export	beaucoup de collectes, transformations, JSON et export morcelé
App/Controller/Integrate.php	parse et insertion	parcours profonds, normalisation répétée, concat SQL
App/Library/Extraction2.php	extraction / lecture	participe au coût global metadata et aux scénarios d'extraction dense
tests/Controller/AspirateurCollectionCpuTest.php	test de gain transparent	verrouille le cache `SHOW DATABASES`

